

TLS / HTTPS

1. Overview

TLS (Transport Layer Security) is the cryptographic protocol that secures data in transit across a network, providing **confidentiality, integrity, and authentication** for a connection between two parties.

HTTPS is simply HTTP layered on top of TLS — the same familiar web protocol, but with the connection itself secured.

TLS is the practical convergence point of nearly every other topic in this guide series: it relies on **certificates, PKI, digital signatures, asymmetric key exchange, and symmetric bulk encryption** all working together.

TLS at a Glance

Property	Detail
Purpose	Secure a network connection — confidentiality, integrity, authentication
Relationship to HTTPS	HTTPS = HTTP running over a TLS-secured connection
Current versions in use	TLS 1.2, TLS 1.3 (TLS 1.0/1.1 are deprecated and considered insecure)
Predecessor	SSL (Secure Sockets Layer) — now obsolete; "SSL" is still used colloquially, but modern deployments exclusively use TLS

2. What TLS Actually Provides

TLS provides three primary security properties:

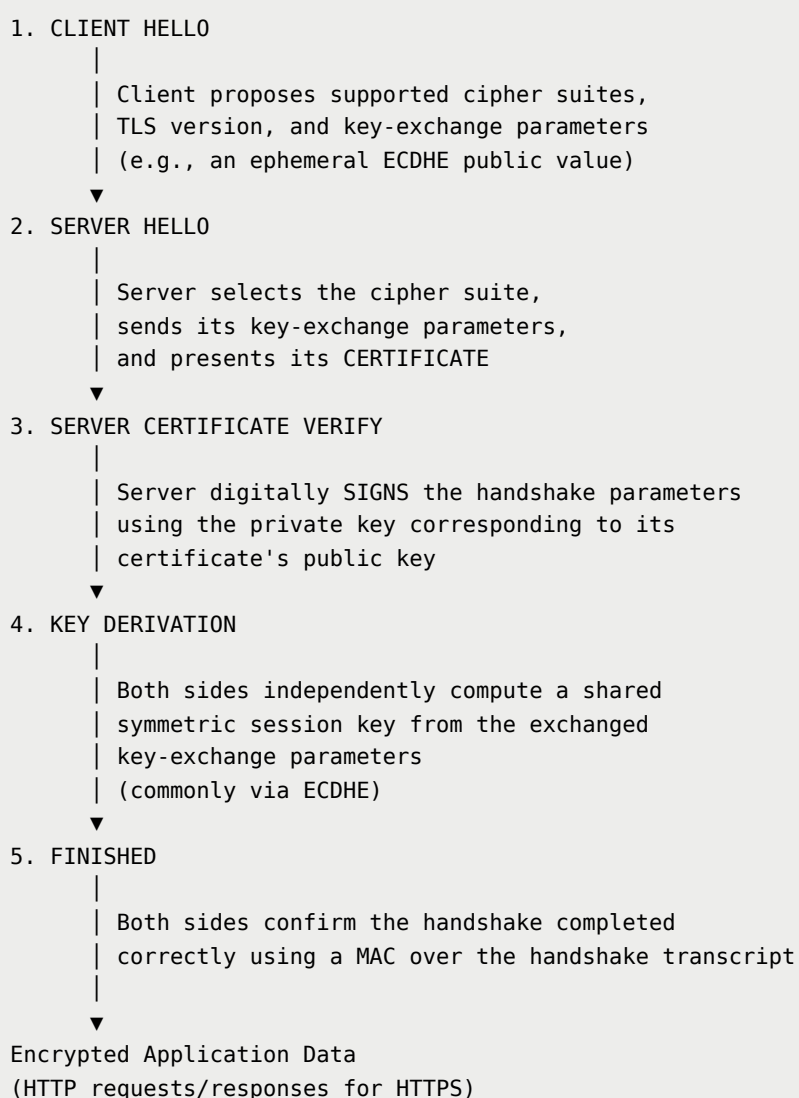
1. **Confidentiality** — data exchanged after the handshake is encrypted using a symmetric cipher, typically AES, and is unreadable to anyone intercepting the traffic.
2. **Integrity** — any tampering with data in transit is detectable, typically via an authenticated encryption mode such as AES-GCM or a separate MAC.
3. **Authentication** — the server, and optionally the client, proves its identity via a certificate chained back to a trusted CA.

Key point: TLS protects the connection by combining encryption, integrity protection, and identity authentication.

3. The TLS Handshake — High-Level Flow (TLS 1.3)

TLS 1.3 significantly simplified and sped up the handshake relative to TLS 1.2, removing legacy/insecure options and reducing the number of round trips required.

TLS 1.3 Handshake Flow



Step-by-Step

Step	Process	Description
------	---------	-------------

1	CLIENT HELLO	Client proposes supported cipher suites and TLS version and sends key-exchange parameters, such as an ephemeral ECDHE public value.
2	SERVER HELLO	Server selects the cipher suite, sends its key-exchange parameters, and presents its certificate.
3	SERVER CERTIFICATE VERIFY	Server digitally signs the handshake parameters using the private key corresponding to its certificate's public key, proving it genuinely holds that private key.
4	KEY DERIVATION	Both sides independently compute a shared symmetric session key from the exchanged key-exchange parameters, commonly via ECDHE.
5	FINISHED	Both sides confirm the handshake completed correctly using a MAC over the handshake transcript. Encrypted application data then begins flowing under the derived symmetric session key.

4. Why Both Asymmetric and Symmetric Cryptography Are Used

TLS is a practical example of **hybrid cryptography**, combining the strengths of both approaches.

Asymmetric Cryptography

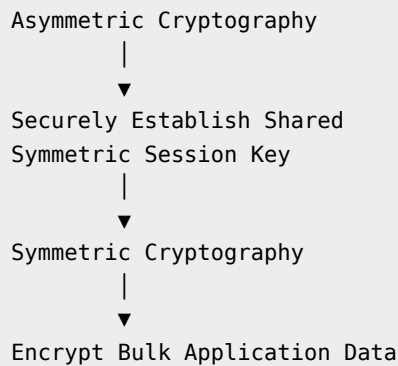
RSA/ECC is used during the handshake:

- For the server's certificate signature
- For the key-exchange mechanism itself
- Because it solves the key distribution problem without requiring a pre-shared secret

Symmetric Cryptography

AES is used for the actual bulk application data after the handshake completes because it is vastly faster than asymmetric operations and is well suited to encrypting large volumes of traffic.

The Core Idea



Key point: The handshake uses asymmetric cryptography only long enough to securely establish a shared symmetric session key. The much faster symmetric cipher then protects the bulk of the communication.

5. Certificate Verification During the Handshake

When the client receives the server's certificate, it performs several checks before proceeding.

Certificate Validation Checks

- The certificate's signature chains validly back to a **CA recognized by the client's trust store**.
- The certificate has **not expired** — the current date falls within its validity period.
- The certificate has **not been revoked**, checked via CRL, OCSP, or OCSP stapling.
- The certificate's **Subject or Subject Alternative Name (SAN)** matches the hostname the client is actually trying to connect to.

What Happens If Validation Fails?

A failure at any of these checks should cause the client to **abort the connection** or, in a browser, present a prominent security warning rather than silently proceeding.

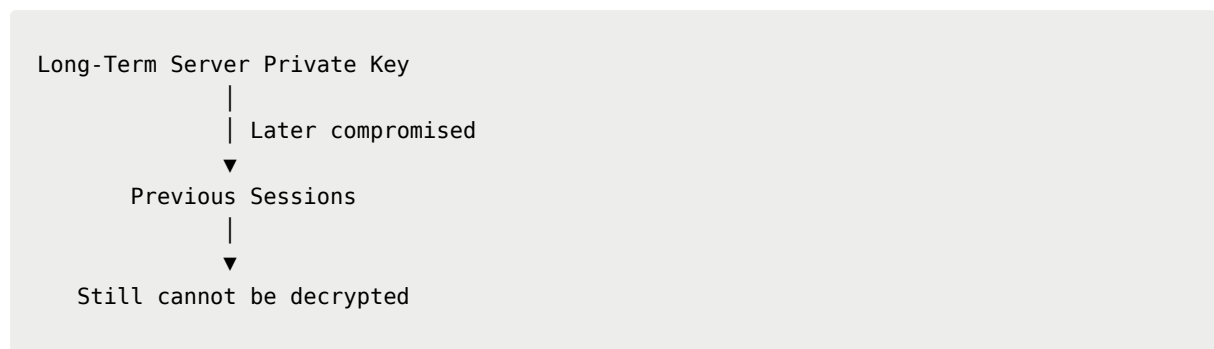
6. Forward Secrecy

Modern TLS deployments use **ephemeral key exchange**, specifically ECDHE rather than static/non-ephemeral Diffie-Hellman or RSA key transport, to provide **forward secrecy**.

How Forward Secrecy Works

A fresh, temporary key pair is generated for each session and discarded afterward.

Therefore:



Even if a server's long-term private key is later compromised, previously captured encrypted traffic from past sessions cannot retroactively be decrypted because:

- The session keys that protected the traffic no longer exist.
- The session keys were not derivable from the long-term key alone.

Key point: Forward secrecy limits the damage caused by a later compromise of a server's long-term private key.

7. TLS 1.3 vs. TLS 1.2 — Key Differences

Aspect	TLS 1.2	TLS 1.3
Handshake round trips	Typically 2	Reduced to 1 (with 0-RTT resumption available for repeat connections)
Legacy/weak cipher support	Permitted many now-deprecated options, such as RC4 and static RSA key exchange	Removed entirely — only modern, forward-secret cipher suites are supported
Renegotiation	Supported (historically a source of vulnerabilities)	Removed

Overall design goal	Broad backward compatibility	Simplicity, security-by-default, reduced attack surface
----------------------------	------------------------------	---

8. Common Misconceptions

Misconception 1: "HTTPS Means the Site Is Safe or Legitimate"

HTTPS only guarantees that the **connection is encrypted** and that the certificate's identity claim was validated at whatever level the issuing CA performed.

For example, many certificates use **Domain Validation (DV)**, which may only establish domain control.

It says nothing about the trustworthiness of the site's actual content or operator.

■ A site can use HTTPS and still be malicious.

Misconception 2: "SSL" and "TLS" Are Interchangeable

Although the terms are often used casually, all actual **SSL protocol versions are obsolete and insecure**.

Modern "SSL certificates" or "SSL connections" are, in practice, using **TLS**.

Misconception 3: A Valid Certificate Protects User Privacy From the Server

A valid certificate proves server identity and helps secure the connection, but it does not control what the server does with the data after receiving it.

TLS protects data **in transit between the client and server**.

It says nothing about how the server subsequently:

- Handles the data
- Stores the data
- Shares the data

9. Typical Use Cases

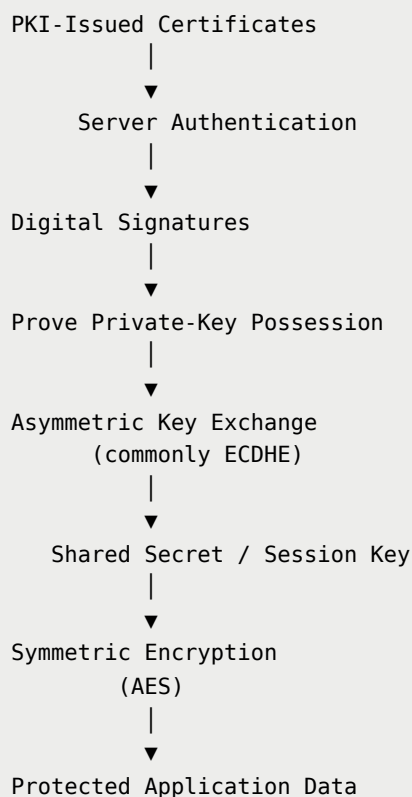
TLS is commonly used for:

- **HTTPS** — securing standard web traffic; now the overwhelming default for the modern web.
- **Email transmission** — SMTP, IMAP, and POP3 over TLS.
- **VPN protocols** that layer on TLS, such as OpenVPN.
- **API authentication and data transport** — including mutual TLS (mTLS) for service-to-service authentication in microservice and zero-trust architectures.
- **Database connection encryption** between application servers and backend databases.

10. Summary

TLS is the protocol that operationalizes nearly every other topic in this series into a single working system.

It uses:



TLS Brings Together

1. **PKI-issued, CA-signed certificates** — authenticate the server.
2. **Digital signatures** — prove private-key possession during the handshake.
3. **Asymmetric key exchange**, commonly ECDHE — establishes a shared secret without transmitting it directly.
4. **Symmetric encryption**, such as AES — protects the bulk of the actual data exchanged.

Bottom line: HTTPS is simply the everyday name for **HTTP running over a TLS-secured connection**.